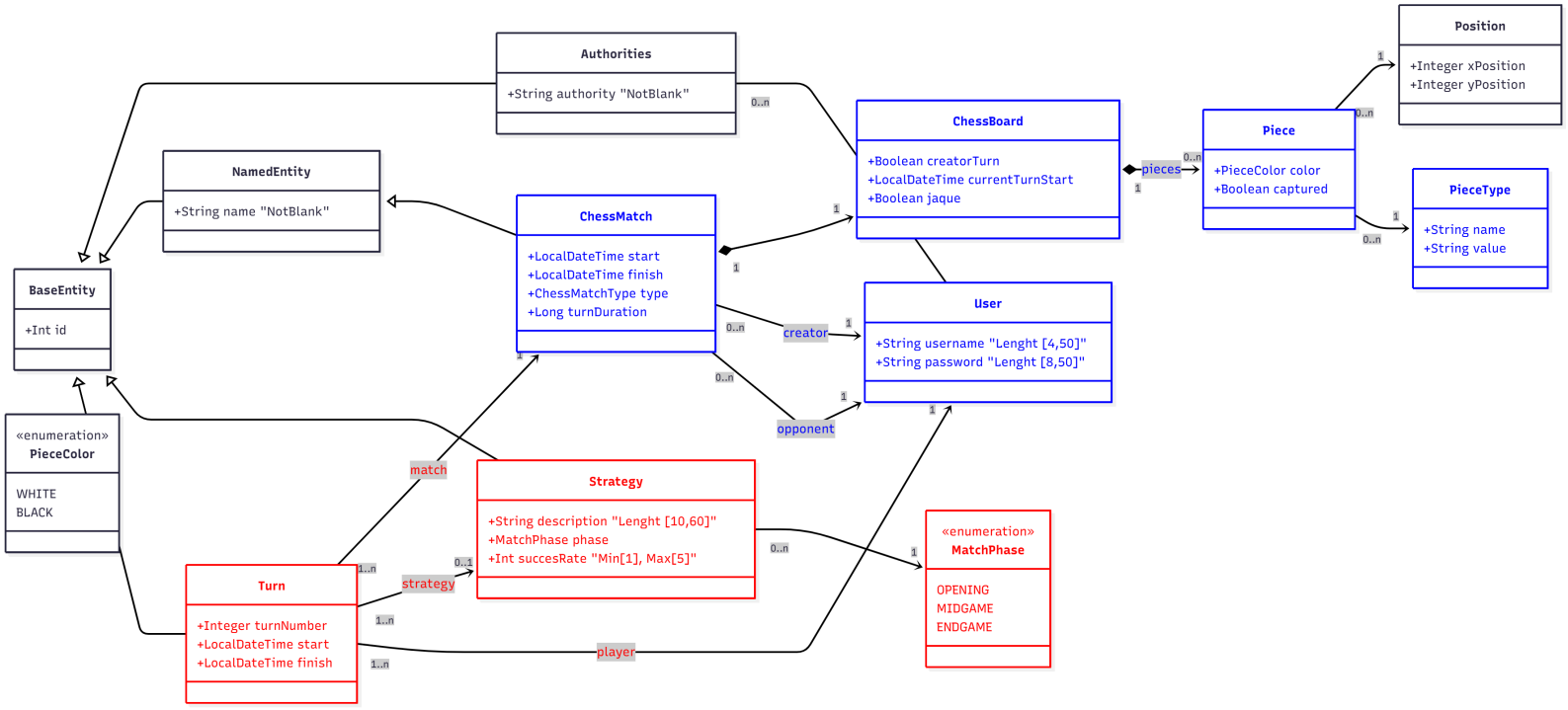
# Control práctico de DP1 2024-2025 (Tercera Convocatoria Oct. 2025)

## Enunciado

En este ejercicio, añadiremos la funcionalidad de un diario de estrategias aplicadas en el juego del ajedrez. Concretamente, se proporciona una clase “ChessMatch” que representa las partidas que se juegan, y que tiene asociada una instancia de la clase “ChessBoard” que representa el estado del tablero para dicha partida, por lo que tendrá asociada un conjunto de instancias de la clase “Piece”. Además, tendremos las clase “Strategy”, que representa a una estrategia adoptada por un jugador en un turno determinado de una partida, por ejemplo “realizar la apertura Italiana en el primer turno”, o “atacar el centro en el turno 15”. Para expresar la fase de la partida en la que se aplica la estrategia se usa el enumerado “MatchPhase”. Los jugadores de una partida van registrando la estrategia a utilizar.

El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:




Las clases para las que realizaremos el mapeo objeto-relacional como entidades JPA se han señalado en rojo. Las clases en azul son clases que se proporcionan ya mapeadas, pero con las que se trabajará durante el control de laboratorio.

Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias. Si desea ver el resultado que arrojarían las pruebas en backend, puede ejecutarlas (bien mediante su entorno de desarrollo favorito, bien mediante el comando “*mvnw test*” en la carpeta raíz del proyecto). Cada ejercicio correctamente resuelto valdrá un punto, el número de casos de prueba de cada ejercicio puede variar entre uno y otro y la nota se calculará en base al porcentaje de casos de prueba que pasan. Por ejemplo, si pasan la mitad (50%) de los casos de prueba de un ejercicio, en lugar de un punto usted obtendrá un 0.5.

Para comenzar el control debe aceptar la tarea de este control práctico a través del siguiente enlace:

https://classroom.github.com/a/yhaJGtD3

Al aceptar dicha tarea, se creará un repositorio único individual para usted, debe usar dicho repositorio para realizar el control práctico. Debe entregar la actividad en EV asociada al control check proporcionando como texto la dirección url de su repositorio personal. Recuerde que además debe entregar su solución del control.

**La entrega de su solución al control se realizará mediante un único comando "*git push*" a su repositorio individual.** Recuerde que debe hacer push antes de cerrar sesión en la computadora y abandonar el aula, de lo contrario, su intento se evaluará como no presentado. Su primera tarea en este control será clonar (recuerde que si va a usar los equipos del aula para realizar el control necesitará usar un token de autenticación de GitHub como clave, tiene un documento de ayuda a la configuración en el propio repositorio del control). A continuación, deberá importar el proyecto en su entorno de desarrollo favorito y comenzar los ejercicios abajo listados. Al importar el proyecto, el mismo puede presentar errores de compilación. No se preocupe, si existen, dichos errores irán despareciendo conforme usted vaya implementando los distintos ejercicios del control.

**Nota importante 1:** No modifique los nombres de las clases ni la signatura (nombre, tipo de respuesta y parámetros) de los métodos proporcionados como material de base para el control. Las pruebas que se usan para la evaluación dependen de que las clases y los métodos tengan la estructura y nombres proporcionados. Si los modifica probablemente no pueda hacer que pasen las pruebas, y obtendrá una mala calificación.

**Nota importante 2:** No modifique las pruebas unitarias proporcionadas como parte del proyecto bajo ningún concepto. Aunque modifique las pruebas en su copia local del proyecto, éstas serán restituidas mediante un comando git previamente a la ejecución de las pruebas para la emisión de la nota final, por lo que sus modificaciones en las pruebas no serán tenidas en cuenta en ningún momento.

**Nota importante 3:** Mientras haya ejercicios no resueltos habrá tests que no funcionen y, por tanto, el comando "*mvnw install*" finalizará con error. Esto es normal debido a la forma en la que está planteado el control y no hay que preocuparse por ello. Si se quiere probar la aplicación se puede ejecutar de la forma habitual pese a que "mvnw install" finalice con error.

**Nota importante 4**: La descarga del material de la prueba usando git, y la entrega de su solución con git a través del repositorio GitHub creado a tal efecto forman parte de las competencias evaluadas durante el examen, por lo que no se aceptarán entregas que no hagan uso de este medio, y no se podrá solicitar ayuda a los profesores para realizar estas tareas.

**Nota importante 5**: No se aceptarán como soluciones válidas proyectos cuyo código fuente no compile correctamente o que provoquen fallos al arrancar la aplicación en la inicialización del contexto de Spring. Las soluciones cuyo código fuente no compile o incapaces de arrancar el contexto de Spring serán evaluadas con una nota de 0.

## Test 1 – Creación de las entidades Strategy y Turn y sus repositorios asociados

Modificar la clase "Strategy" para que sea una entidad. Esta clase está alojada en el paquete "es.us.dp1.chess.tournament.strategy", y debe tener los siguientes atributos y restricciones:

- El atributo de tipo entero (Integer) llamado "id" actuará como clave primaria en la tabla de la base de datos relacional asociada a la entidad.
- Un atributo de tipo cadena de caracteres (String) llamado "description" obligatorio (no puede ser nulo), que debe tener una longitud mínima de **10** caracteres y máxima de **60**.[1]
- Un atributo del tipo enumerado "MatchPhase" llamado "phase" que indica la fase en la que se encuentra la partida al registrar la estrategia. Este atributo es obligatorio y debe almacenarse como una cadena en la BD ("OPENING", "MIDGAME", o "ENDGAME").
- El atributo de tipo entero (Integer) llamado "successRate", que representa el grado de éxito conseguido con la estrategia y que de be tomar un valor entre 1 y 5. Este atributo será especificado tras ejecutar la estrategia y por tanto es un atributo opcional que mientras se está ejecutando la estrategia no tendrá ningún valor asociado.

Modificar la clase "Turn" para que sea una entidad. Esta clase está alojada en el paquete "es.us.dp1.chess.tournament.match", y debe tener los siguientes atributos y restricciones:

- El atributo de tipo entero (Integer) llamado "id" actuará como clave primaria en la tabla de la base de datos relacional asociada a la entidad.
- El atributo de tipo entero (Integer) obligatorio llamado "turnNumber" que indicará el número de turno de la partida, siendo un número incremental independientemente del jugador que esté jugando de tal manera que el creador de la partida jugará primero en el turno 1, y el oponente jugará por primera vez en el turno 2. Por tanto el mínimo número permitido de turn será el 1.
- El atributo de tipo instante temporal (LocalDateTime) llamado "startTime", que representa el instante en el que el turno del jugador comienza, y es un atributo obligatorio.
- El atributo de tipo instante temporal (LocalDateTime) llamado "finishTime", que representa el instante en el que el turno del jugador finaliza y es un atributo opcional ya que mientras se está en el turno no tendrá ningún valor asociado.

No modifique por ahora las anotaciones @Transient de las clases. Modificar las interfaces "StrategyRepository" y "TurnRepository" alojadas en los mismos paquetes que "Strategy" y "Turn" respectivamente, para que extiendan CrudRepository. No olvide especificar sus parámetros de tipo.

## Test 2 – Creación de relaciones entre las entidades

Elimine las anotaciones @Transient de los métodos y atributos que las tengan en las entidades creadas en el ejercicio anterior. Se pide crear las siguientes relaciones entre las entidades:
Cree una relación unidireccional desde "Turn" hacia "ChessMatch" que exprese la que aparece en el diagrama UML (mostrado en la primera página de este enunciado), usando el atributo "match". Debe asegurarse de que las relaciones expresan adecuadamente la cardinalidad que muestra el diagrama UML.

Cree una relación unidireccional desde "Turn" hacia "Strategy" que exprese la que aparece en el diagrama UML (mostrado en la primera página de este enunciado), usando el atributo "strategy". Debe asegurarse

[1] Nótese que estas restricciones no coinciden con las de NamedEntity.

de que las relaciones expresan adecuadamente la cardinalidad que muestra el diagrama UML considerando que los atributos pueden ser nulos puesto que la cardinalidad en el extremo navegable es 0..1.

Cree una relación unidireccional desde “Turn” hacia “User” que exprese la que aparece en el diagrama UML (mostrado en la primera página de este enunciado), usando el atributo “player”. Debe asegurarse de que las relaciones expresan adecuadamente la cardinalidad que muestra el diagrama UML.

## Test 3 – Modificación del script de inicialización de la base de datos para incluir tres estrategias y dos turnos

Modificar el script de inicialización de la base de datos, para que se creen las siguientes estrategias:

Strategy 1:

- id: 1
- description: Focus on rapid development and control of the center
- phase: OPENING
- sucessRate: null

Strategy 2:

- id: 2
- description: Prioritize piece safety and solid positioning
- phase: MIDGAME
- sucessRate: null

Strategy 3:

- id: 3
- description: Aim for king safety and pawn structure integrity
- phase: ENDGAME
- sucessRate: null

Turn 1:

- id: 1
- turnNumber: 1
- startTime: 2025-10-20 12:00
- endTime: 2025-10-20 12:01

Turn 2:

- id: 2
- turnNumber: 2
- startTime: 2025-10-20 12:01
- endTime: 2025-10-20 12:02

## Test 4 – Modificación del script de inicialización de la base de datos para relacionar los turnos con las estrategias, las partidas, y los jugadores

Modificar este script de inicialización de la base de datos para que:

- El turno cuyo id es 1 se asocie con:
  - la estrategia cuyo id es 1,
  - la partida cuyo id es 15, y
  - el usuario cuyo id es 4.
- El turno cuyo id es 2 se asocie con:
  - ninguna estrategia,
  - la partida cuyo id es 15, y
  - el usuario cuyo id es 5.

Tenga en cuenta que el orden en que aparecen los INSERT en el script de inicialización de la base de datos es relevante al definir las asociaciones.

## Test 5 – Creación de servicios de gestión de estrategias

Modificar la clase "TurnService", para que sea un servicio Spring de lógica de negocio y proporcione una implementación a los métodos que permita:

1. Obtener todos los turnos almacenadas en la base de datos como una lista usando el repositorio (método findAll).
2. Obtener un turno por Id (método findById).
3. Grabar un turno en la base de datos, teniendo en cuenta que el User al que se le asigna el turno tiene que ser el que está actualmente logado en el sistema y que este sea el que corresponde en el turno actual de la partida. En caso de que no sea un User adecuado para dicho turno se lanzará una excepción de tipo WrongUserAssignmentException (puede usar el método getCurrentTurn del repositorio de turnos para obtener el turno actual y comprobar el User adecuado)

Todos estos métodos **deben ser transaccionales**, pero las anotaciones asociadas deben realizarse a nivel de método, no a nivel de clase**.** No modifique por ahora la implementación del resto de métodos de los servicios.

## Test 6 – Anotar el repositorio de partidas con una consulta compleja

Modificar la consulta personalizada que puede invocarse a través del método "findCapturePoints" del repositorio de partida "MatchRepository" (alojado en el paquete es.us.dp1.chess.tournament.match) que recibe como parámetro un id de "ChessMatch" devuelve una lista de pares (Object[]), que contienen un color y un entero. El objetivo es que calcule el **total de puntos de las piezas capturadas de cada color** en dicha partida.

En otras palabras:

- Si se pasa el color "**BLACK"**, el método debe devolver la suma de los valores de todas las piezas **negras** capturadas en esa partida (es decir, los puntos obtenidos por el jugador blanco).
- Si se pasa el color **WHITE**, debe devolver la suma de los valores de las piezas **blancas** capturadas (los puntos del jugador negro).

Nótese que PieceType tiene un atributo value que representa los puntos que vale esa pieza. Para implementar este ejercicio debe modificar la consulta asociada al método del repositorio llamado "findCapturePoints", sin modificar su signatura (tipo de respuesta, nombre y parámetros).

## Test 7 – Creación del controlador para devolver las piezas capturadas al contrario por parte de un jugador de una partida disponible

Crear un método en el controlador "*ChessMatchController"* (alojada en el paquete es.us.dp1.chess.tournament.match) que permita devolver las piezas que el jugador logado en una partida ha capturado al contrario. El método debe responder a peticiones tipo GET en la URL:

http://localhost:8080/api/v1/matches/X/captures/

El método debe devolver las capturas del usuario logado en el sistema en una partida concreta, donde X es la Id de la partida. Recuerde que la clase UserService tiene un método "findCurrentUser()" que permite obtener el usuario actualmente logado, debe usar dicho método para obtener el usuario actual.

Se debe devolver el código de estado 404 (NOT_FOUND) en los siguientes casos:

- En caso de que no exista una partida con el id especificado el sistema.
- En caso de que no se hayan capturado aún piezas.

En caso de que el usuario logado no sea uno de los jugadores de dicha partida debe devolverse un código de estado 403 (FORBIDDEN). Nótese que se proporcionan excepciones ya creadas para representar estos escenarios, como la clase ResourceNotFoundException o la clase AccessDeniedException.

Para implementar este ejercicio debe modificar tanto la clase ChessMatchService (método getCaptures) como la clase ChessMatchController (añadiendo un método al controlador para responder a la peticiones). Implementando la lógica de negocio y gestión de la comunicación donde corresponda.

## Test 8 – Implementar una prueba para un algoritmo que comprueba el estado de la partida para un movimiento del alfil ("bishop") (concretamente si hay jaque).

Con el objetivo de facilitar la interacción con la página, se ha decidido crear un conjunto de algoritmos que permita comprobar si nuestro rey está en jaque. En este ejercicio crearemos un conjunto de casos de prueba que permitan comprobar que el algoritmo funciona correctamente.

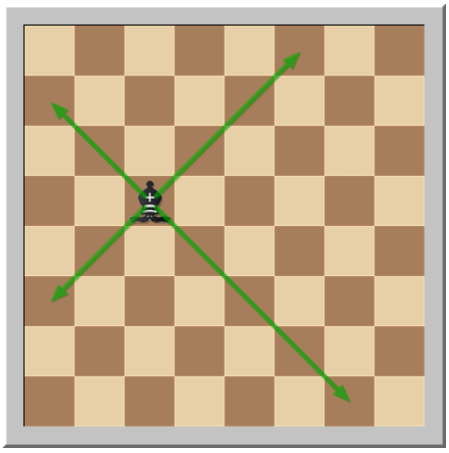
La interfaz del algoritmo está especificada en la interfaz "BishopJaqueDetector" que se encuentra en el paquete "es.us.dp1.chess.tournament.match.jaquedetector", que pretende detectar cuando un alfil contrario está dando jaque a nuestro rey (asumiendo que no hay piezas en la trayectoria que lo impidan). Se proporcionan cuatro implementaciones del algoritmo. Concretamente el método que comprueba el estado de jaque tiene la siguiente signatura: boolean isJaque(int bishopX, int bishopY, int kingX, int kingY). Si se le pasan al algoritmo posiciones inválidas por definición (nótese que los valores de todos los parámetros deberían estar siempre entre 0 y 7 para representar posiciones válidas en el tablero), el algoritmo deber lanzar una excepción de tipo InvalidPositionException.

Modifique la clase de pruebas llamada "BishopJaqueDetectorTest" que se encuentra en la carpeta "scr/test/java/es/us/dp1/chess/tournament/bishopjaquedetector" y especifique tantos métodos con casos de prueba como considere necesarios para validar el correcto funcionamiento del algoritmo. Nótese

que la clase tiene un atributo de tipo BishopJaqueDetector llamado algorithm, use dicho atributo como sujeto bajo prueba en todos sus métodos de prueba.

Su implementación del test **no debe usar mocks, ni anotaciones de pruebas de spring** (@DataJpaTest, @SpringBootTest,etc.), **ni tests parametrizados, y todos los métodos anotados con @Test deben ser sin parámetros**.

Recuerde que el movimiento del alfil en ajedrez tiene la siguiente estructura:



## Test 9 – Creación de un componente básico React de información de fichas capturadas en un tablero

Modificar el componente React proporcionado en el fichero "frontend/src/matches/board.js" para que muestre los detalles de capturas en el tablero de una partida para el jugador actual. Para ello debe hacer uso de la API lanzando una petición tipo GET contra la URL api/v1/matches/X/captures para obtener la lista de piezas capturadas por el jugador logado. Dicha API devuelve un resultado similar al siguiente:


```
[
  { "id": 5000,
    "position": null,
    "type": {
      "name": "QUEEN",
      "value": 9
    },
    "color": "BLACK",
    "captured": true  },
  {
    "id": 5001,
    "position": null,
    "type": {
      "name": "ROOK",
      "value": 5
    },
    "color": "BLACK",
    "captured": true
  }
```

]

Este componente debe mostrar los datos siguientes datos del tablero:

- Un título de nivel 1 cuyo contenido sea el texto “Board info”
- Un título de nivel 3 cuyo contenido sea “Captures”
- Mostrar el total de puntos de las capturas obtenidas en un div. Por ejemplo para el caso mostrado arriba debería mostrar el texto: “Total Points: 14”.

Para poder lanzar esta prueba y comprobar su resultado puede colocarse en la carpeta de frontend y ejecutar el comando npm test y pulsar 'a’ en el menú de comandos de jest. Nótese que previamente debe haber lanzado al menos una vez el comando npm install para que todas las librerías de node estén instaladas.

## Test 10 – Creación de un componente para mostrar por pantalla en una partida las piezas capturadas al jugador contrario.

Complete el código del componente boardDetail, para hacer que se muestre un lista no ordenada (<ul>) con las fichas capturadas . Concretamente cada ficha  capturada debe mostrarse en un item de lista (<li>) mostrando primero el color de la ficha, un guión y el nombre del tipo de ficha. Por ejemplo, para la respuesta mostrada en el ejercicio anterior se mostraría:

- BLACK-QUEEN
- BLACK-ROOK

Para poder lanzar esta prueba y comprobar su resultado puede colocarse en la carpeta de frontend y ejecutar el comando npm test y pulsar 'a’ en el menú de comandos de jest. Nótese que previamente debe haber lanzado al menos una vez el comando npm install para que todas las librerías de node estén instaladas.